BIO392: Introduction to R

Feifei Xia

17.04.2026

Course Outline

Getting Familiar with R and R studio

Getting started with R syntax and Objects

Building Graphics in R

Basic Setup

Install R

https://cran.r-project.org/

Install Rstudio

https://posit.co/products/open-source/rstudio

Rstudio Interface

RStudio File Edit Code View Plots Session Build Debug Profile Tools Window Help 93% Ven. 10:56 Diana Marek

~/TrainingSIB/Courses_2017/First_Steps_R_Oct2017 - RStudio

Go to file/function | Addins | First_Steps_R_Oct2017

first_script.R | test.R | examples.R | mice_study.R

Source on Save | Run | Source

```
1  #### My first script #####
2  #### October 2017   #####
3
4
5  # list workspace
6  ls()
7
8  # Reset R'brain
9  rm(list=ls())
10
11 # check wd
12 getwd()
13
14 #set wd
15 setwd("/Users/dmarek/EducationSIB/Courses_2016/First_Steps_R_June2016/R_intro_course")
16
17 # confirm wd
18 getwd()
19
20 #load packages if needed (to do every time you launch your R session)
21 #library("boot")
22 #library("lattice")
```

2:26 | October 2017 | R Script

Editor

Console | Terminal

~/TrainingSIB/Courses_2017/First_Steps_R_Oct2017/

```
R est un logiciel libre livré sans AUCUNE GARANTIE.
Vous pouvez le redistribuer sous certaines conditions.
Tapez 'license()' ou 'licence()' pour plus de détails.

R est un projet collaboratif avec de nombreux contributeurs.
Tapez 'contributors()' pour plus d'information et
'citation()' pour la façon de le citer dans les publications.

Tapez 'demo()' pour des démonstrations, 'help()' pour l'aide
en ligne ou 'help.start()' pour obtenir l'aide au format HTML.
Tapez 'q()' pour quitter R.

[Workspace loaded from ~/TrainingSIB/Courses_2017/First_Steps_R_Oct2017/.RData]

>
```

Console, terminal

Environment | History | Connections

Import Dataset | List

Global Environment

Workspace, history

Data	
mice_data	50 obs. of 3 variables
mice_weight_HFD	29 obs. of 3 variables
mids	num [1:2, 1] 0.7 1.9
Values	
mean_weight_diet	num [1:2(1d)] 28.7 37.1
mean_weight_genotype	num [1:2(1d)] 33.7 33.4
n_weight_diet	int [1:2(1d)] 21 29
n_weight_genotype	int [1:2(1d)] 24 26
sd_weight_diet	num [1:2(1d)] 2.61 5
sd_weight_genotype	num [1:2(1d)] 4.69 6.92

Files | Plots | Packages | Help | Viewer

New Folder | Delete | Rename | More

Home > TrainingSIB > Courses_2017 > First_Steps_R_Oct2017 > course_datasets

Name	Size	Modified
..		
class.csv	402 B	Mar 6, 2016, 6:33 PM
etubiol.csv	2.4 KB	Mar 7, 2016, 12:13 PM
mammals_survey.csv	1.2 MB	Jan 30, 2017, 7:41 AM
melanoma_data.txt	5.9 KB	Jan 30, 2017, 7:41 AM
mice_data.csv	632 B	Feb 28, 2016, 4:15 PM
my_data_frame.csv	262 B	Mar 2, 2015, 4:49 PM
pigs.csv	206 B	Mar 6, 2016, 6:32 PM
smoker.csv	4.6 KB	Feb 26, 2015, 12:23 PM
snp.csv	618 B	Mar 6, 2016, 9:19 PM

File explorer, plots, packages, help

R variables

Variables are containers for storing data values.

- A variable is created the moment you first assign a value to it.
- To assign a value to a variable, use the <- sign. To output (or print) the variable value, type the variable name or use print() function.

Example:

```
course <- "BIO392"

course # auto-print the value of the course variable

print(course) # print the value of the course variable
```

In other programming language, it is common to use = as an assignment operator. In R, we can use both = and <- as assignment operators. However, <- is preferred in most cases because the = operator can be forbidden in some context in R.

R data types

- numeric: 10.5, 55, 787
- integer: 1L, 55L, 100L, where the letter "L" declares this as an integer
- complex: 9 + 3i, where "i" is the imaginary part
- character (a.k.a. string): "k", "R is exciting", "FALSE", "11.5"
- logical (a.k.a. boolean): TRUE or FALSE

We can use the class() function to check the data type of a variable

Example:

```
# integer
x <- 1000L
class(x)
```

R operators

Operators are used to perform operations on variables and values.

- Arithmetic operators: x+y

  + (Addition) - (Subtraction) * (Multiplication) / (Division) ^ (Exponent) %% (Remainder from division) %/% (Integer Division)

- Assignment operators: x <- 1

  <- <<- (a global assigner)

- Comparison operators

  == (Equal) != (Not equal) > (Greater than) < (Less than) >= (Greater than or equal to) <= (Less than or equal to)

- Logical operators

  & Element-wise Logical AND operator. It returns TRUE if both elements are TRUE

  && Logical AND operator - Returns TRUE if both statements are TRUE

  | Element-wise Logical OR operator. It returns TRUE if one of the elements is TRUE

  || Logical OR operator. It returns TRUE if one of the statement is TRUE.

  ! Logical NOT - returns FALSE if statement is TRUE

- Miscellaneous operators

  : Creates a series of numbers in a sequence e.g. x <- 1:10

  %in% Find out if an element belongs to a vector e.g. x %in% y

  %*% Matrix Multiplication e.g. x <- Matrix1 %*% Matrix2

Conditional execution & loops

If statements

- if

```
a <- 33
b <- 200
if (b > a) {
  print("b is greater than a")
}
```

- else if

```
a <- 33
b <- 33
if (b > a) {
  print("b is greater than a")
} else if (a == b) {
  print ("a and b are equal")
}
```

- if else

```
a <- 200
b <- 33
if (b > a) {
  print("b is greater than a")
} else if (a == b) {
  print("a and b are equal")
} else {
  print("a is greater than b")
}
```

- AND OR condition

```
a <- 200
b <- 33
c <- 500
if (a > b & c > a) {
  print("Both conditions are true")
}
```

Conditional execution & loops

For / While loop

While loop

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
}
```

- break

```
i <- 1
while (i < 6) {
  print(i)
  i <- i + 1
  if (i == 4) {
    break
  }
}
```

- next

```
i <- 0
while (i < 6) {
  i <- i + 1
  if (i == 3) {
    next
  }
  print(i)
}
```

For loop

```
for (x in 1:10) {
  print(x)
}
```

support break and next statements

R functions

To create a function, use the function() keyword:

```
my_function <- function(x) { # create a function with the name my_function
   5 * x
}
my_function(3) # call the function named my_function with the argument x equals to 3
```

To let a function return a result, use the return() function:

```
my_function <- function(x) { # create a function with the name my_function
   return(5 * x)
}
```

R Data Structures

Vectors

a list of items that are of the same type.

```
# Create a vector of strings
fruits <- c("banana", "apple", “orange”)
```

- To find out how many items a vector has

```
length(fruits)
```

- To sort items in a vector alphabetically or numerically

```
numbers <- c(13, 3, 5, 7, 20, 2)
sort(numbers) # 2  3  5  7 13 20
```

- Access vectors

```
# Access the first item (banana)
fruits[1]
# Access the first and third item (banana and orange)
fruits[c(1, 3)]
# Access the first two items (banana and apple)
fruits[1:2]
```

- Change an Item

```
# Change "banana" to "pear"
fruits[1] <- “pear”
```

- Remove an Item

```
# remove “orange”
newfruits <- fruits[-3]
```

- Add an Item

```
append(fruits, "strawberry")
```

- Repeat Vectors

```
repeat_each <- rep(c(1,2,3), each = 3) # 1,1,1,2,2,2,3,3,3
repeat_times <- rep(c(1,2,3), times = 3) # 1,2,3,1,2,3,1,2,3
repeat_indepent <- rep(c(1,2,3), times = c(5,2,1)) # 1,1,1,1,1,2,2,3
```

- Check if Item Exists

```
"apple" %in% fruits
```

- Generating Sequenced Vectors

```
numbers <- 1:10 # 1,2,3,4,5,6,7,8,9,10
numbers <- seq(from = 0, to = 100, by = 20) # 0  20  40  60  80 100
```

R Data Structures

Lists

a list of items that are of different types.

```
# create a list
thislist <- list("apple", "banana", 2, TRUE)
```

- Most manipulations on lists are similar with that on vectors
- Some exceptions:
  - sort() is unavailable
  - Access

```
# access a sublist
thislist[1]
# retrieve content in the sublist
thislist[[1]]
```

Matrices

a two dimensional data set with columns and rows.

```
# Create a matrix
thismatrix <- matrix(c(1,2,3,4,5,6), nrow = 3, ncol = 2)
```

- Number of rows and columns

```
dim(thismatrix) # 3 2
length(thismatrix) # 6
nrow(thismatrix) # 3
ncol(thismatrix) # 2
```

- Access

```
# Access the element in the first row, the second column
thismatrix[1, 2]
# Access the second row
thismatrix[2,]
# Access the second column
thismatrix[,2]
# Access the first two rows
thismatrix[c(1,2),]
```

- Add rows and columns

```
# add column
newmatrix <- cbind(thismatrix, c(7,8,9))
# add row
newmatrix <- rbind(thismatrix,c(7,8))
```

- Remove rows and columns

```
#Remove the first row and the first column
thismatrix <- thismatrix[-c(1), -c(1)]
```

R Data Structures

Arrays

compared to matrices, arrays can have more than two dimensions.

```
# create an array
multiarray <- array(c(1:24), dim = c(4, 3, 2))
```

Data frames

data displayed in a format as a table. Different columns can be different types of data, but each column should have the same type of data.

```
# Create a data frame
Data_Frame <- data.frame (
  Training = c("Strength", "Stamina", "Other"),
  Pulse = c(100, 150, 120),
  Duration = c(60, 30, 45)
)
```

- Most manipulations on lists are similar with that on matrices
- Some exceptions:
  - Summarize

```
# Summarize values in each column of the data frame
summary(Data_Frame)
```

  - Access

```
# Access the first column
Data_Frame[1]
Data_Frame[["Training"]]
Data_Frame$Training
```

  - length()

```
length(Data_Frame) # 3
```

R plotting

Points

```
# two points in the diagram, one at position (1, 3) and one in position (8, 10)
plot(c(1, 8), c(3, 10))
```

Lines

```
# a line that connects the two points in a diagram
plot(c(1, 8), c(3, 10), type="l")
```

Pie charts

```
# Create a vector of pies
x <- c(10,20,30,40)

pie(x)
```

Bar charts

```
# x-axis values
x <- c("A", "B", "C", "D")

# y-axis values
y <- c(2, 4, 6, 8)

barplot(y, names.arg = x)
```

Histograms

```
# For reproducibility
set.seed(123)
# 100 random values from a normal distribution
data <- rnorm(100, mean = 5, sd = 1)

hist(data)
```

Box pots

```
set.seed(123)
data <- data.frame(
  Group = rep(c("A", "B", "C"),each = 10),
  Value = c(rnorm(10, mean = 5, sd = 1),
            rnorm(10, mean = 7, sd = 1.5),
            rnorm(10, mean = 6, sd = 1))
)

boxplot(Value ~ Group, data = data)
```

Advanced: ggplot2 package (cheatsheet: https://github.com/rstudio/cheatsheets/blob/main/data-visualization.pdf)

Exercise

Complete exercise-code.r by replacing all ???